I don't suggest using brew install elixir or other global package managers. There's too much that inevitably goes wrong. This is especially true if you're working on multiple projects that may update at different times.

Before we wrap up this chapter, let's write a bit of code!

Write Your First Elixir Code

While Elixir is a compiled language, it has an interactive console that makes it easy to quickly experiment with code. We'll use Elixir's console—IEx—to write and execute basic code. IEx (Interactive Elixir) is the equivalent of IRB (Interactive Ruby) in the Ruby world.

IEx is commonly used to quickly prototype code locally, although it's a bit more powerful than that. One of the convenient techniques that you can use—and you will later in the book—is to run your web server and interactive console at the same time. This allows you to directly experiment with and modify a running application.

It's easy to start an IEx session. Run the following command on your command line. If you see an error, then go back to the previous section and make sure that Elixir and Erlang/OTP are properly installed.

```
$ iex
Erlang/OTP 24 [erts-12.0.3]...

Interactive Elixir (1.12.3) - press Ctrl+C to exit (type h() ENTER for help)
iex(1)>
```

Enter a few lines of code to see things working:

```
iex(1)> 1 + 1
2

iex(2)> IO.puts("A string")
A string
:ok

iex(3)> adder = fn (a, b) -> a + b end
#Function<43.40011524/2 in :erl_eval.expr/5>

iex(4)> adder.(17, 25)
42
```

When you're done, hit Ctrl + C, followed by A, and then ENTER. This is the easiest way to exit out of the IEx session.

Okay, I'll admit this isn't *that* exciting yet. But you have written your first lines of Elixir! As you enter each line, IEx evaluates it, executes it, and outputs the result on the next line—just like IRB does.